

details, not on the main conceptions. Not everyone understands the necessary scientific role of such experiments in computational living, nor that standard experimental techniques cannot provide the same information. How else to explain, for example, the calls for controlled experimental validation that speech understanding will be useful to computer science? When that experiment of style is finally performed there will be no doubt at all. No standard experiment will be necessary. Indeed, none could have sufficed.

As an example related to the present paper, I have spent some effort recently in describing what Herb Simon and I have called the “*Physical symbol system hypothesis*” [26, 29]. This hypothesis identifies a class of systems as embodying the essential nature of symbols and as being the necessary and sufficient condition for a generally intelligent agent. Symbol systems turn out to be universal computational systems, viewed from a different angle. For my point here, the important feature of this hypothesis is that it grew out of the practice in AI—out of the development of list processing languages and Lisp, and out of the structure adopted in one AI program after another. We in AI were led to an adequate notion of a symbol by our practice. In the standard catechism of science, this is not how great ideas develop. Major ideas occur because great scientists discover (or invent) them, introducing them to the scientific community for testing and elaboration. But here, working scientists have evolved a new major scientific concept, under partial and alternative guises. Only gradually has it acquired its proper name.

The notions of knowledge and representation about to be presented also grow out of our practice. At least, so I assert. That does not give them immunity from criticism, for in listening for these lessons I may have a tin ear. But in so far as they are wanting, the solution lies in more practice and more attention to what emerges there as pragmatically successful. Of course, the message will be distorted by many things, e.g., peculiar twists in the evolution of computer science hardware and software, our own limitations of view, etc. But our practice remains a source of knowledge that cannot be obtained from anywhere else. Indeed, AI as a field is committed to it. If it is fundamentally flawed, that will just be too bad for us. Then, other paths will have to be found from elsewhere to discover the nature of intelligence.

3. The Knowledge Level

I am about to propose the existence of something called the *knowledge level*, within which knowledge is to be defined. To state this clearly, requires first reviewing the notion of computer systems levels.

3.1. Computer system levels

Fig. 2 shows the standard hierarchy, familiar to everyone in computer science. Conventionally, it starts at the bottom with the *device level*, then up to the

circuit level, then the *logic level*, with its two sublevels, *combinatorial and sequential circuits*, and the *register-transfer level*, then the *program level* (referred to also as the *symbolic level*) and finally, at the top, the *configuration level* (also called the *PMS* or *Processor-Memory-Switch level*). We have drawn the configuration level to one side, since it lies directly above both the symbol level and the register-transfer level.

The notion of levels occurs repeatedly throughout science and philosophy, with varying degrees of utility and precision. In computer science, the notion is quite precise and highly operational. Table 1 summarizes its essential attributes. A level consists of a *medium* that is to be processed, *components* that provide primitive processing, *laws of composition* that permit components to be assembled into *systems*, and *laws of behavior* that determine how system behavior depends on the component behavior and the structure of the system. There are many variant instantiations of a given level, e.g., many programming systems and machine languages and many register-transfer systems.⁴

Each level is defined in two ways. First, it can be defined autonomously, without reference to any other level. To an amazing degree, programmers need not know logic circuits, logic designers need not know electrical circuits, managers can operate at the configuration level with no knowledge of programming, and so forth. Second, each level can be reduced to the level below. Each aspect of a level—medium, components, laws of composition and behavior—can be defined in terms of systems at the level next below. The *architecture* is the name we give to the register-transfer level system that defines a symbol (programming) level, creating a machine language and making it run as described in the programmers manual for the machine. Neither of these two definitions of a level is the more fundamental. It is essential that they both exist and agree.

Some intricate relations exist between and within levels. Any instantiation of a level can be used to create any instantiation of the next higher level. Within each level, systems hierarchies are possible, as in the subroutine hierarchy at the programming level. Normally, these do not add anything special in terms of the computer system hierarchy itself. However, as we all know, at the program level it is possible to construct any instantiation within any other instantiation (modulo some rigidity in encoding one data structure into another), as in creating new programming languages.

There is no need to spin out the details of each level. We live with them every day and they are the stuff of architecture textbooks [1], machine manuals and digital component catalogues, not research papers. However, it is noteworthy how radically the levels differ. The medium changes from electrons and magnetic domains at the device level, to current and voltage at the circuit level,

⁴Though currently dominated by electrical circuits, variant circuit level instantiations also exist, e.g., fluidic circuits.

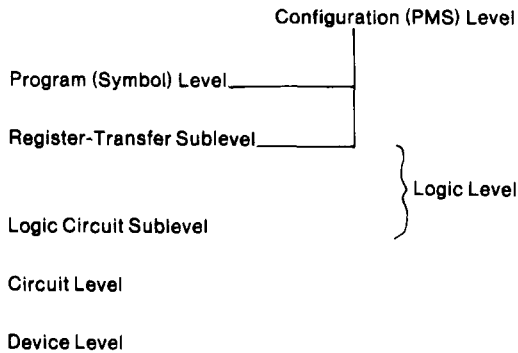


FIG. 2. Computer system levels.

to bits at the logic level (either single bits at the logic circuit level or bit vectors at the register-transfer level), to symbolic expressions at the symbol level, to amounts of data (measured in data bits) at the configuration level. System characteristics change from continuous to discrete processing, from parallel to serial operation, and so on.

Despite this variety, all levels share some common features. Four of these, though transparently obvious, are important to us.

Point 1. Specification of a system at a level always determines completely a definite behavior for the system at that level (given initial and boundary conditions).

Point 2. The behavior of the total system results from the local effects of each component of the system processing the medium at its inputs to produce its outputs.

Point 3. The immense variety of behavior is obtained by system structure, i.e., by the variety of ways of assembling a small number of component types (though perhaps a large number of instances of each type).

TABLE 1. Defining aspects of a computer system level

Aspects	Register-transfer level	Symbol level
Systems	Digital systems	Computers
Medium	Bit vectors	Symbols, expressions
Components	Registers	Memories
	Functional units	Operations
Composition laws	Transfer path	Designation, association
Behavior laws	Logical operations	Sequential interpretation

Point 4. The medium is realized by state-like properties of matter, which remain passive until changed by the components.

Computer systems levels are not simply levels of abstraction. That a system has a description at a given level does not necessarily imply it has a description at higher levels. There is no way to abstract from an arbitrary electronic circuit to obtain a logic-level system. There is no way to abstract from an arbitrary register-transfer system to obtain a symbol-level system. This contrasts with many types of abstraction which can be uniformly applied, and thus have a certain optional character (as in abstracting away from the visual appearance of objects to their masses). Each computer system level is a *specialization* of the class of systems capable of being described at the next lower level. Thus, it is a priori open whether a given level has any physical realizations.

In fact, computer systems at all levels are realizable, reflecting indirectly the structure of the physical world. But more holds than this. Computer systems levels are realized by *technologies*. The notion of a technology has not received the conceptual attention it deserves. But roughly, given a specification of a particular system at a level, it is possible to construct by *routine* means a physical system that realizes that specification. Thus, systems can be obtained to specification within limits of time and cost. It is not possible to invent arbitrarily additional computer system levels that nestle between existing levels. Potential levels do not become technologies, just by being thought up. Nature has a say in whether a technology can exist.

Computer system levels are *approximations*. All of the above notions are realized in the real world only to various degrees. Errors at lower levels propagate to higher ones, producing behavior that is not explicable within the higher level itself. Technologies are imperfect, with constraints that limit the size and complexity of systems that can actually be fabricated. These constraints are often captured in design rules (e.g., fan-out limits, stack-depth limits, etc), which transform system design from routine to problem solving. If the complexities become too great, the means of system creation no longer constitute a technology, but an arena of creative invention.

We live quite comfortably with imperfect system levels, especially at the extremes of the hierarchy. At the bottom, the device level is not complete, being used only to devise components at the circuit level. Likewise, at the top, the configuration level is incomplete, not providing a full set of behavioral laws. In fact, it is more nearly a pure level of abstraction than a true system level. This accounts for both symbol level and register transfer level systems having configuration (PMS) level abstractions (see [2] for a PMS approach to the register-transfer level).

These levels provide ways of describing computer systems; they do not provide ways of describing their environments. This may seem somewhat unsatisfactory, because a level does not then provide a general closed description of an entire universe, which is what we generally expect (and get) from a

level of scientific description in physics or chemistry. However, the situation is understandable enough. System design and analysis requires only that the interface between the environment and the system (i.e., the inner side of the transducers) be adequately described in terms of each level, e.g., as electrical signals, bits, symbols or whatever. Almost never does the universe of system plus environment have to be modeled in toto, with the structure and dynamics of the environment described in the same terms as the system itself. Indeed, in general no such description of the environment in the terms of a given computer level exists. For instance, no register-transfer level description exists of the airplane in which an airborne computer resides. Computer system levels describe the internal structure of a particular class of systems, not the structure of a total world.

To sum up, computer system levels are a reflection of the nature of the physical world. They are not just a point of view that exists solely in the eye of the beholder. This reality comes from computer system levels being genuine specializations, rather than being just abstractions that can be applied uniformly.

3.2. A new level

I now propose that there does exist yet another system level, which I will call the *knowledge level*. It is a true systems level, in the sense we have just reviewed. The thrust of this paper is that distinguishing this level leads to a simple and satisfactory view of knowledge and representation. It dissolves some of the difficulties and confusions we have about this aspect of artificial intelligence.

A quick overview of the knowledge level, with an indication of some of its immediate consequences, is useful before entering into details.

The system at the knowledge level is the *agent*. The components at the knowledge level are *goals*, *actions*, and *bodies*. Thus, an agent is composed of a set of actions, a set of goals and a body. The medium at the knowledge level is *knowledge* (as might be suspected). Thus, the agent processes its knowledge to determine the actions to take. Finally, the behavior law is the *principle of rationality*: Actions are selected to attain the agent's goals.

To treat a system at the knowledge level is to treat it as having some knowledge and some goals, and believing it will do whatever is within its power to attain its goals, in so far as its knowledge indicates. For example:

- “She knows where this restaurant is and said she'd meet me here. I don't know why she hasn't arrived.”

- “Sure, he'll fix it. He knows about cars.”

- “If you know that $2 + 2 = 4$, why did you write 5?”

The knowledge level sits in the hierarchy of systems levels immediately above the symbol level, as Fig. 3 shows. Its components (actions, goals, body)

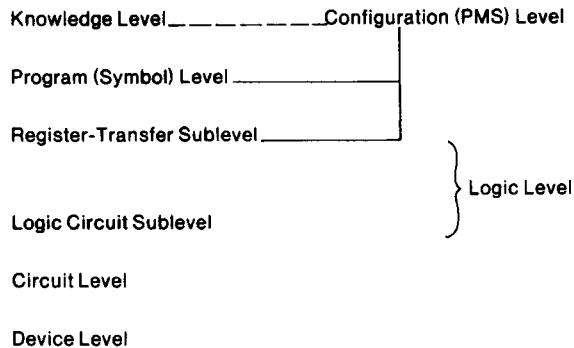


FIG. 3. New version of computer system levels.

and its medium (knowledge) can be defined in terms of systems at the symbol level, just as any level can be defined by systems at the level one below. The knowledge level has been placed side by side with the configuration level. The gross anatomical description of a knowledge-level system is simply (and only) that the agent has as parts bodies of knowledge, goals and actions. They are all connected together in that they enter into the determination of what actions to take. This structure carries essentially no information; the specification at the knowledge level is provided entirely by the content of the knowledge and the goals, not by any structural way they are connected together. In effect, the knowledge level can be taken to have a degenerate configuration level.

As is true of any level, although the knowledge level can be constructed from the level below (i.e., the symbol level), it also has an autonomous formulation as an independent level. Thus, knowledge can be defined independent of the symbol level, but can also be reduced to symbol systems.

As the Fig. 3 makes evident, the knowledge level is a separate level from the symbol level. Relative to the existing view of the computer systems hierarchy, the symbol level has been split in two, one aspect remaining as the symbol level proper and another aspect becoming the knowledge level. The description of a system as exhibiting intelligent behavior still requires both levels, as we shall see. Intelligent systems are not to be described exclusively in terms of the knowledge level.

To repeat the final remark of the prior section: Computer system levels really exist, as much as anything exists. They are not just a point of view. Thus, to claim that the knowledge level exists is to make a scientific claim, which can range from dead wrong to slightly askew, in the manner of all scientific claims. Thus, the matter needs to be put as a hypothesis:

The Knowledge Level Hypothesis. *There exists a distinct computer systems level, lying immediately above the symbol level, which is characterized by knowledge as the medium and the principle of rationality as the law of behavior.*

Some preliminary feeling for the nature of knowledge according to this hypothesis can be gained from the following remarks.

- Knowledge is intimately linked with rationality. Systems of which rationality can be posited can be said to have knowledge. It is unclear in what sense other systems can be said to have knowledge.

- Knowledge is a competence-like notion, being a potential for generating action.⁵

- The knowledge level is an approximation. Nothing guarantees how much of a system's behavior can be viewed as occurring at the knowledge level. Although extremely useful, the approximation is quite imperfect, not just in degree but in scope.

- Representations exist at the symbol level, being systems (data structures and processes) that realize a body of knowledge at the knowledge level.

- Knowledge serves as the specification of what a symbol structure should be able to do.

- Logics are simply one class of representations among many, though uniquely fitted to the analysis of knowledge and representation.

4. The Details of the Knowledge Level

We begin by defining the knowledge level autonomously, i.e., independently of lower system levels. Table 1 lists what is required, though we will take up the various aspects in a different order: first, the structure, consisting of the system, components and laws for composing systems; second, the laws of behavior (the law of rationality); and third, the medium (knowledge). After this we will describe how the knowledge level reduces to the symbol level.

Against the background of the common features of the familiar computer system levels listed earlier (Section 3.1), there will be four surprises in how the knowledge level is defined. These will stretch the notion of system level somewhat, but will not break it.

4.1. The structure of the knowledge level

An agent (the system at the knowledge level), has an extremely simple structure, so simple there is no need even to picture it.

First, the agent has some *physical body* with which it can act in the environment (and be acted upon). We talk about the body as if it consists of a set of *actions*, but that is only for simplicity. It can be an arbitrary physical system with arbitrary modes of interaction with its environment. Though this

⁵How almost interchangeable the two notions might be can be seen from a quotation from Chomsky [9, p. 315]: "In the past I have tried to avoid, or perhaps evade the problem of explicating the notion 'knowledge of language' by using an invented technical term, namely the term 'competence' in place of 'knowledge'".

body can be arbitrarily complex, its complexity lies *external* to the system described at the knowledge level, which simply has the power to evoke the behavior of this physical system.

Second, the agent has a *body* of knowledge. This body is like a memory. Whatever the agent knows at some time, it continues to know. Actions can add knowledge to the existing body of knowledge. However, in terms of structure, a body of knowledge is extremely simple compared to a memory, as defined at lower computer system levels. There are no structural constraints to the knowledge in a body, either in capacity (i.e., the amount of knowledge) or in how the knowledge is held in the body. Indeed, there is no notion of how knowledge is held (*encoding* is a notion at the symbol level, not knowledge level). Also, there are no well-defined structural properties associated with access and augmentation. Thus, it seems preferable to avoid calling the body of knowledge a memory. In fact, referring to a 'body of knowledge', rather than just to 'knowledge', is hardly more than a manner of speaking, since this body has no function except to be the physical component which has the knowledge.

Third, and finally, the agent has a *set of goals*. A goal is a body of knowledge of a state of affairs in the environment. Goals are structurally distinguished from the main body of knowledge. This permits them to enter into the behavior of the agent in a distinct way, namely, that which the organism strives to realize. But, except for this distinction, goal components are structurally identical to bodies of knowledge. Relationships exist between goals, of course, but these are not realized in the structure of the system, but in knowledge.

There are no laws of composition for building a knowledge level system out of these components. An agent always has just these components. They all enter directly into the laws of behavior. There is no way to build up complex agents from them.

This complete absence of significant structure in the agent is the first surprise, running counter to the common feature at all levels that variety of behavior is realized by variety of system structure (Section 3.1, Point 3). This is not fortuitous, but is an essential feature of the knowledge level. The focus for determining the behavior of the system rests with the knowledge, i.e., with the *content* of what is known. The internal structure of the system is defined exactly so that nothing need be known about it to predict the agent's behavior. The behavior is to depend only what the agent knows, what it wants and what means it has for interacting physically with the environment.

4.2. The principle of rationality

The behavioral law that governs an agent, and permits prediction of its behavior, is the rational principle that knowledge will be used in the service of

goals.⁶ This can be formulated more precisely as follows:

Principle of rationality. *If an agent has knowledge that one of its actions will lead to one of its goals, then the agent will select that action.*

This principle asserts a connection between knowledge and goals, on the one hand, and the selection of actions on the other, without specification of any mechanism through which this connection is made. It connects all the components of the agent together directly.

This direct determination of behavior by a global principle is the second surprise, running counter to the common feature at all levels that behavior is determined bottom-up through the local processing of components (Section 3.1, Point 2). Such global principles are not incompatible with systems whose behavior is also describable by mechanistic causal laws, as testified by various global principles in physics, e.g., Fermat's principle of least time in geometrical optics, or the principle of least effort in mechanics. The principles in physics are usually optimization (i.e., extremum) principles. However, the principle of rationality does not have built into it any notion of optimal or best, only the automatic connection of actions to goals according to knowledge.

Under certain simple conditions, the principle as stated permits the calculation of a system's trajectory, given the requisite initial and boundary conditions (i.e., goals, actions, initial knowledge, and acquired knowledge as determined by actions). However, the principle is not sufficient to determine the behavior in many situations. Some of these situations can be covered by adding *auxiliary* principles. Thus, we can think of an *extended principle of rationality*, building out from the *central* or *main* principle, given above.

To formulate additional principles, the phrase *selecting an action* is taken to mean that the action becomes a member of a candidate set of actions, the *selected set*, rather than being the action that actually occurs. When all principles are taken into account, if only one action remains in the selected set, that action actually taken is determined; if several candidates remain, then the action actually taken is limited to these possibilities. An action can actually be taken only if it is physically possible, given the situation of the agent's body and the resources available. Such limits to action will affect the actions selected only if the agent has knowledge of them.

The main principle is silent about what happens if the principle applies for more than one action for a given goal. This can be covered by the following auxiliary principle.

⁶This principle is not intended to be directly responsive to all the extensive philosophical discussion on rationality, e.g., the notion that rationality implies the ability for an agent to give *reasons* for what it does.

[16,39]. It need not be retraced here. The exhibition of the first few elementary extensions can serve to indicate the total development to be taken in search of a principle of rationality that always determines the action to be taken.

Complete retracing is not necessary because the path does not lead, even ultimately, to the desired principle. No such principle exists.⁸ Given an agent in the real world, there is no guarantee at all that his knowledge will determine which actions realize which goals. Indeed, there is no guarantee even that the difficulty resides in the incompleteness of the agent's knowledge. There need not exist any state of knowledge that would determine the action.

The point can be brought home by recalling Frank Stockton's famous short story, "The lady or the tiger?" [38]. The upshot has the lover of a beautiful princess caught by the king. He now faces the traditional ordeal of judgment in this ancient and barbaric land: In the public arena he must choose to open one of two utterly indistinguishable doors. Behind one is a ferocious tiger and death; behind the other a lovely lady, life and marriage. Guilt or innocence is determined by fate. The princess alone, spurred by her love, finds out the secret of the doors on the night before the trial. To her consternation she finds that the lady behind the door is to be her chief rival in loveliness. On the judgment day, from her seat in the arena beside the king, she indicates by a barely perceptible nod which door her lover should open. He, in unhesitating faithfulness, goes directly to that door. The story ends with a question. Did the princess send her lover to the lady or the tiger?

Our knowledge-level model of the princess, even if it were to include her complete knowledge, would not tell us which she chose. But the failure of determinacy is not the model's. Nothing says that multiple goals need be compatible, nor that the incompatibility be resolvable by any higher principles. The dilemma belongs to the princess, not to the scientist attempting to formulate an adequate concept of the knowledge level. That she resolved it is clear, but that her behavior in doing so was describable at the knowledge level does not follow.

This failure to determine behavior uniquely is the third surprise, running counter to the common feature at all levels that a system is a determinate machine (Section 3.1, Point 1). A complete description of a system at the program, logic or circuit level yields the trajectory of the system's behavior over time, given initial and boundary conditions. This is taken to be one of its important properties, consistent with being the description of a deterministic (macro) physical system. Yet, *radical* incompleteness characterizes the knowl-

⁸An adequate critical recounting of this intellectual development is not possible here. Formal systems (utility fields) can be constructed that appear to have the right property. But they work, not by connecting actions with goals via knowledge of the task environment, but by positing of the agent a complete set of goals (actually, preferences) that directly specify all action selections over all combinations of task states (or probabilistic options over task states). Such a move actually abandons the enterprise.

edge level. Sometimes behavior can be predicted by the knowledge level description; often it cannot. The incompleteness is not just a failure in certain special situations or in some small departures. The term *radical* is used to indicate that entire ranges of behavior may not be describable at the knowledge level, but only in terms systems at a lower level (namely, the symbolic level). However, the necessity of accepting this incompleteness is an essential aspect of this level.

4.3. The nature of knowledge

The ground is now laid to provide the definition of knowledge. As formulated so far, knowledge is defined to be the medium at the knowledge level, something to be processed according to the principle of rationality to yield behavior. We wish to elevate this into a complete definition:

Knowledge. *Whatever can be ascribed to an agent, such that its behavior can be computed according to the principle of rationality.*

Knowledge is to be characterized entirely *functionally*, in terms of what it does, not *structurally*, in terms of physical objects with particular properties and relations. This still leaves open the requirement for a physical structure for knowledge that can fill the functional role. In fact, that key role is never filled directly. Instead, it is filled only indirectly and approximately by *symbol systems* at the next lower level. These are total systems, not just symbol structures. Thus, knowledge, though a medium, is embodied in no medium-like passive physical structure.

This failure to have the medium at the knowledge level be a state-like physical structure is the fourth surprise, running counter to the common feature at all levels of a passive medium (Section 3.1, Point 4). Again, it is an essential feature of the knowledge level, giving knowledge its special abstract and competence-like character. One can see on the blackboard a symbolic expression (say a list of theorem names). Though the actual seeing is a tad harder, yet there is still no difficulty seeing this same expression residing in a definite set of memory cells in a computer. The same is true at lower levels—the bit vectors at the register-transfer level—marked on the blackboard or residing in a register as voltages. Moreover, the medium at one level plus additional static structure defines the medium at the next level up. The bit plus its organization into registers provides the bit-vector; collections of bit vectors plus functional specialization to link fields, type fields, etc., defines the symbolic expression. All this fails at the knowledge level. The knowledge cannot so easily be seen, only imagined as the result of interpretive processes operating on symbolic expressions. Moreover, knowledge is not just a collec-

tion of symbolic expressions plus some static organization; it requires both processes and data structures.

The definition above may seem like a reverse, even perverse, way of defining knowledge. To understand it—to see how it works, why it is necessary, why it is useful, and why it is effective—we need to back off and examine the situation in which knowledge is used.

How it works

Fig. 4 shows the situation, which involves an *observer* and an *agent*. The observer treats the agent as a system at the knowledge level, i.e., ascribes knowledge and goals to it. Thus, the observer knows the agent's knowledge (K) and goals (G), along with his possible actions and his environment (these latter by direct observation, say). In consequence, the observer can make predictions of the agent's actions using the principle of rationality.

Assume the observer ascribes correctly, i.e., the agent behaves as if he has knowledge K and goals G . What the agent really has is a symbol system, S , that permits it to carry out the calculations of what actions it will take, because it has K and G (with the given actions in the given environment).

The observer is itself an agent, i.e., is describable at the knowledge level. There is no way in this theory of knowledge to have a system that is not an agent have knowledge or ascribe knowledge to another system. Hence, the observer has all this knowledge (i.e., K' , consisting of knowledge K , knowledge that K is the agent's knowledge, knowledge G , knowledge that these are the agent's goals, knowledge of the agent's actions etc.). But what the observer really has, of course, is a symbol system, S' , that lets it calculate actions on the basis of K , goals, etc., i.e., calculate what the agent would do if it had K and G .

Thus, as the figure shows, each agent (the observer and the observed) has

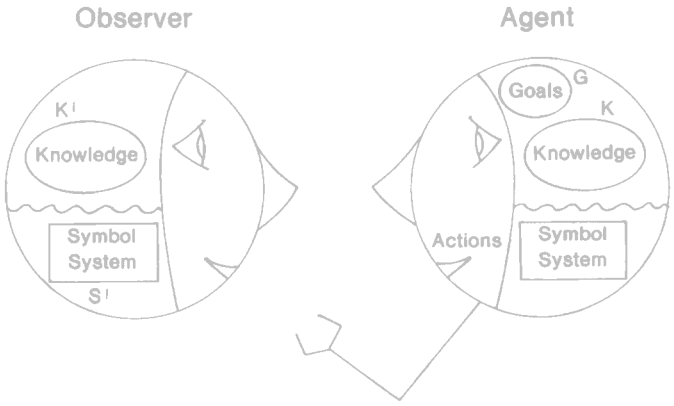


FIG. 4. The situation in which knowledge is used.